

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1103

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL3).*

Assessment Tool Weightage: 40%

Dr S Ashok Kumar

QUESTIONS: 8

Total Marks:40

Annexure A

CONCEPT MAPPING

Code of Conduct:

*I **_SYED SABIRA(192373044)**_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Syed sabira

1. Object-Oriented Design in Smart Healthcare Systems

Introduction

A Smart Healthcare System is a software application that manages various hospital operations such as patient registration, doctor management, appointment scheduling, medical records, prescriptions, laboratory services, billing, and emergency care. The system aims to improve healthcare services by reducing manual work, increasing efficiency, and ensuring accurate patient information. Object-Oriented Design (OOD) is widely used in developing healthcare systems because it organizes the software into objects that represent real-world entities. This approach makes the software easier to develop, maintain, and expand.

Object-Oriented Concepts

Class

A class is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) of an object.

Examples of classes in a Smart Healthcare System:

- Patient
- Doctor
- Appointment
- Medical Record
- Hospital
- Prescription
- Laboratory Report

Each class represents a real-world entity in the healthcare system.

Object

An object is an instance of a class. It contains actual values and performs specific functions.

Examples:

- Patient: John
- Doctor: Dr. Kumar
- Appointment: Appointment No. 101
- Prescription: Prescription No. 5001

Objects interact with each other to perform healthcare operations.

Encapsulation

Encapsulation combines data and methods into a single unit while protecting sensitive information from unauthorized access. Patient information such as medical history, diagnosis, laboratory reports, and billing details are stored as private data. Access is provided only through authorized methods.

Advantages of Encapsulation

- Protects confidential patient information.
- Prevents unauthorized data modification.
- Improves software security.
- Maintains data integrity.
- Simplifies maintenance.

Inheritance

Inheritance allows one class to inherit the properties and methods of another class. It promotes code reuse and reduces duplication.

Example:

- Person (Parent Class)
 - Patient
 - Doctor
 - Nurse

Common attributes such as Name, Age, Gender, Address, and Contact Number are defined in the Person class and inherited by other classes.

Advantages of Inheritance

- Reduces code duplication.
- Improves code reusability.
- Simplifies software development.
- Makes future enhancements easier.

Polymorphism

Polymorphism allows the same method to perform different tasks depending on the object that calls it.

Example:

The method **calculateBill()** behaves differently for:

- General Patient
- Insurance Patient
- Emergency Patient

Each calculates the hospital bill using different rules.

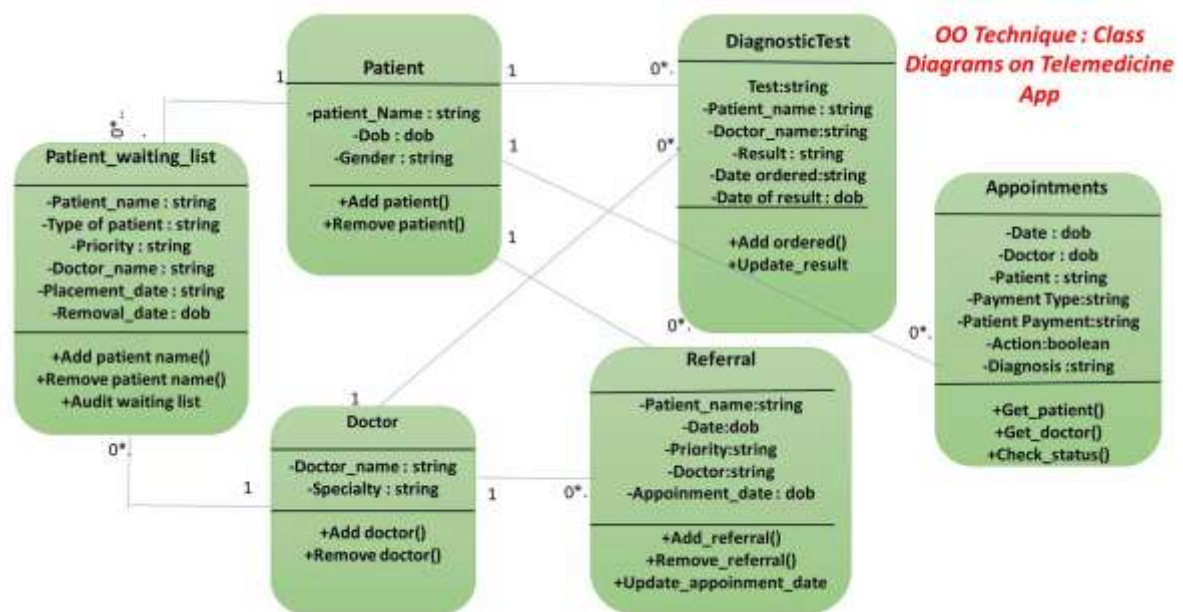
Advantages of Polymorphism

- Improves flexibility.
- Supports runtime decision-making.
- Simplifies code management.

- Makes the system extensible.

Benefits of Object-Oriented Design

- High scalability.
- Improved security.
- Better code reuse.
- Easy maintenance.
- Reduced development time.
- Simplified debugging.
- Increased software reliability.



Analysis

Object-Oriented Design enables Smart Healthcare Systems to manage complex hospital operations efficiently. Each module is independent, making it easier to update or replace without affecting other modules. New features such as Telemedicine, Online Consultation, Electronic Health Records (EHR), AI-based diagnosis, and Health Monitoring can be added without redesigning the entire system. Encapsulation protects sensitive patient information, inheritance reduces duplicate code, and polymorphism provides flexibility in implementing different healthcare services.

Conclusion

Object-Oriented Design provides a strong foundation for Smart Healthcare Systems. It improves software quality, security, maintainability, scalability, and flexibility while reducing development cost and improving healthcare service delivery.

2. Applying Encapsulation and Abstraction in Online Banking

Introduction

An Online Banking System allows customers to perform banking transactions through the internet. Services include checking account balances, transferring funds, paying bills, viewing transaction history, applying for loans, and managing accounts. Since banking systems handle highly confidential financial information, object-oriented principles such as Encapsulation and Abstraction are essential for ensuring security, privacy, and simplicity.

Abstraction

Abstraction hides unnecessary implementation details and presents only the essential features to users. Customers interact only with banking services without knowing the internal processing.

Customers can:

- Deposit money.
- Withdraw money.
- Transfer funds.
- View balance.
- View transaction history.

Customers do not see:

- Database operations.
- Password encryption.
- Authentication algorithms.
- Transaction processing.
- Server communication.

Advantages of Abstraction

- Reduces system complexity.
- Improves user experience.
- Makes software easier to understand.
- Supports flexible implementation.
- Simplifies maintenance.

Encapsulation

Encapsulation bundles data and methods into a single class while protecting sensitive information. Variables such as account balance, PIN, account number, and customer details are kept private and accessed only through authorized methods.

Example

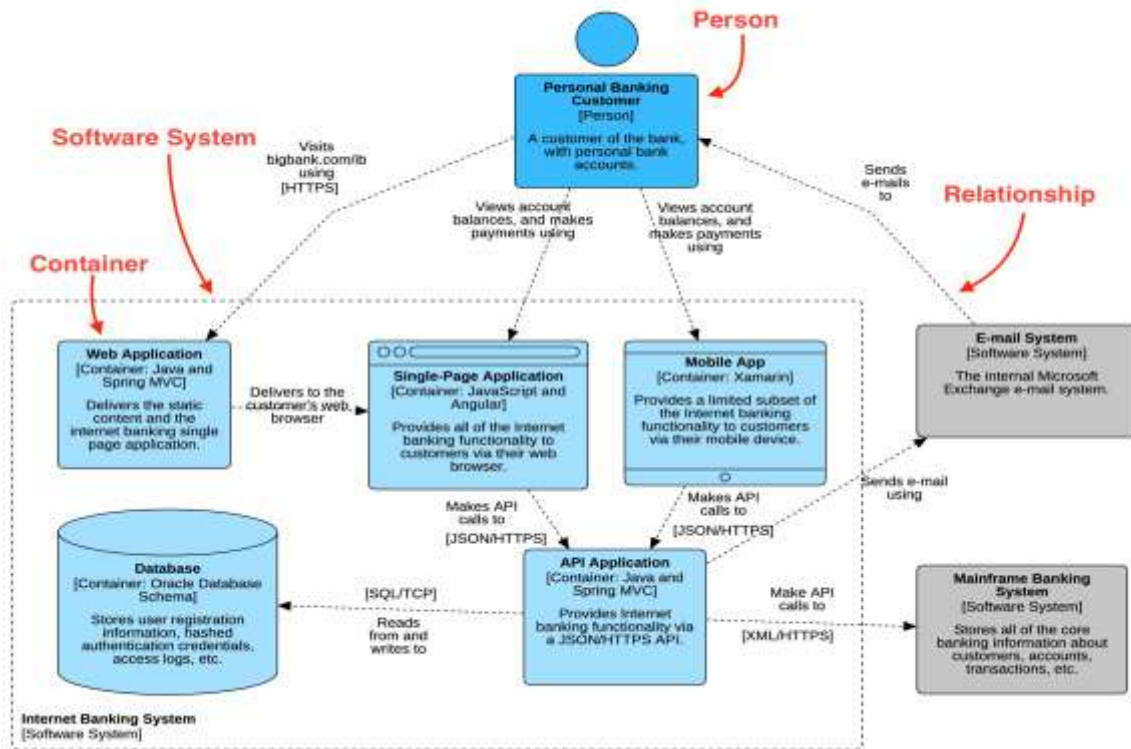
The account balance cannot be modified directly. Customers must use methods such as deposit(), withdraw(), and transfer().

Advantages of Encapsulation

- Protects sensitive banking information.

- Prevents unauthorized access.
- Maintains data consistency.
- Improves security.
- Simplifies maintenance.

[Container] Internet Banking System
The container diagram for the Internet Banking System



Comparison between Abstraction and Encapsulation

Abstraction	Encapsulation
Hides implementation details	Hides internal data
Focuses on functionality	Focuses on security
Uses abstract classes and interfaces	Uses access modifiers
Simplifies user interaction	Protects confidential information

Analysis

Abstraction allows customers to use banking services without understanding complex internal operations. Encapsulation protects confidential customer information such as account balance, passwords, and transaction records. Together, these concepts improve software security, reduce system complexity, and enhance user satisfaction.

Conclusion

Encapsulation and Abstraction are fundamental object-oriented principles that ensure Online Banking Systems are secure, efficient, reliable, and user-friendly. They simplify software development while protecting sensitive customer data.

3. Modeling Object Relationships in a University Management System

Introduction

A University Management System is used to manage academic activities such as student admission, faculty management, department administration, course registration, examinations, attendance, and result processing. Proper object relationship modeling improves software organization and supports efficient system development. The three major object relationships are Association, Aggregation, and Composition.

Association

Association represents a relationship where two objects communicate but remain independent of each other.

Example

Students enroll in Courses.

A student can study multiple courses, and each course can have many students.

If a student leaves the university, the course still exists.

Advantages

- Flexible relationship.
- Independent object existence.
- Easy maintenance.
- Supports many-to-many relationships.

Aggregation

Aggregation represents a weak "Has-A" relationship.

Example

A Department contains Faculty Members.

Faculty members belong to a department but can be transferred to another department without being deleted.

Advantages

- Weak ownership.
- Easy object reuse.
- Flexible system design.
- Better maintainability.

Composition

Composition represents a strong "Has-A" relationship.

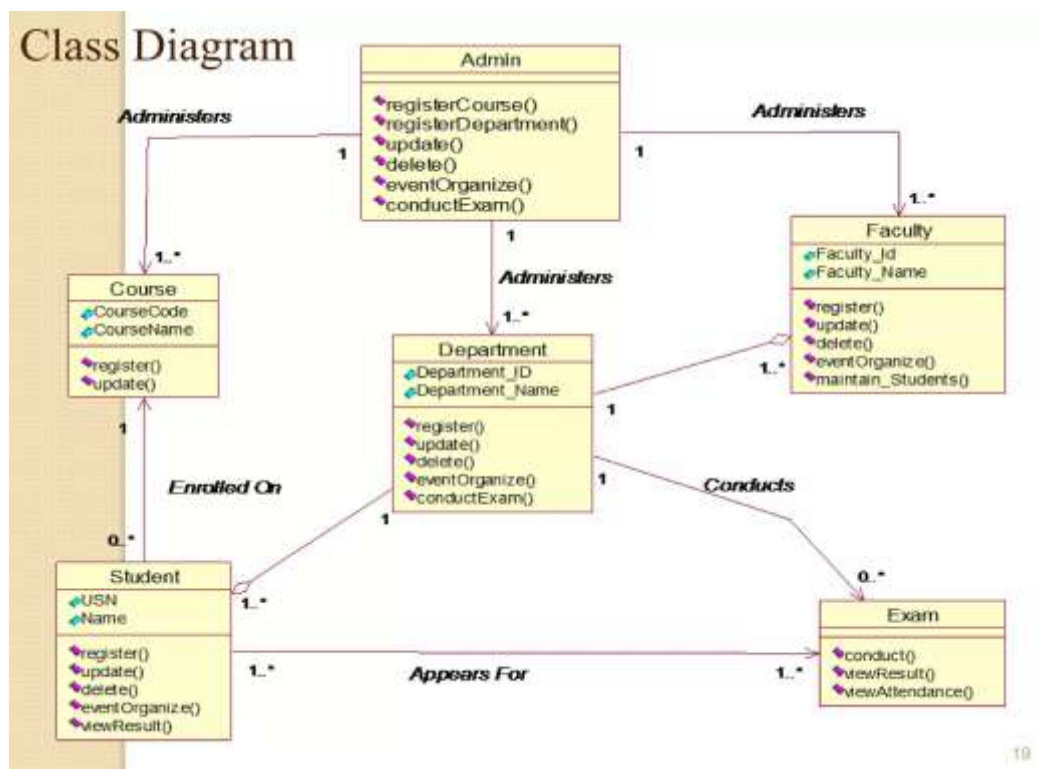
Example

University contains Departments.

If the University is removed, its Departments also cease to exist.

Advantages

- Strong ownership.
- Better data consistency.
- Improved object lifecycle management.
- Simplified system organization.



Comparison

Association	Aggregation	Composition
Independent objects	Weak ownership	Strong ownership
Objects exist separately	Objects partially depend	Objects fully depend
Loose coupling	Moderate coupling	Strong coupling

Benefits of Proper Relationship Modeling

- Improves flexibility.
- Enhances code reuse.
- Simplifies software maintenance.

- Supports scalability.
- Reduces software complexity.
- Improves software quality.

Analysis

Proper relationship modeling helps developers understand how objects interact within the system. It reduces unnecessary dependencies, simplifies modifications, and improves software maintainability. The University Management System becomes easier to expand with additional features such as online learning, digital libraries, and research management.

Conclusion

Association, Aggregation, and Composition are essential object-oriented relationships that improve software flexibility, maintainability, scalability, and overall system performance.

4. Importance of SDLC in E-Commerce Development

Introduction

An E-Commerce Platform allows customers to browse products, place orders, make online payments, and track deliveries. Developing such a system requires a structured process called the Software Development Life Cycle (SDLC). SDLC provides a systematic approach for planning, designing, developing, testing, deploying, and maintaining software. It ensures that the final product is reliable, secure, and meets customer requirements.



Phases of SDLC

1. Requirement Analysis

The development team gathers customer requirements and identifies system functionalities such as user registration, product search, shopping cart, online payment, inventory management, and order tracking.

2. System Design

System architects prepare the software architecture, database design, user interface, module structure, and security mechanisms. A proper design serves as the blueprint for development.

3. Coding (Implementation)

Developers write the source code according to the design documents. Modules such as Login, Product Management, Shopping Cart, Payment Gateway, and Order Management are implemented.

4. Testing

Testing ensures that the software functions correctly and meets customer requirements. Different testing methods include Unit Testing, Integration Testing, System Testing, Performance Testing, and Security Testing.

5. Deployment

After successful testing, the software is deployed on production servers and made available to customers. Users can now access the E-Commerce platform through web browsers or mobile applications.

6. Maintenance

Maintenance is the final phase where bugs are fixed, security updates are applied, performance improvements are made, and new features are added based on customer feedback.

Importance of SDLC

- Provides a structured development process.
- Improves software quality.
- Reduces project risks.
- Detects errors early.
- Saves development cost and time.
- Enhances customer satisfaction.
- Supports future enhancements.
- Improves software reliability and security.

Analysis

Each SDLC phase contributes significantly to the success of an E-Commerce application. Requirement Analysis ensures that customer needs are understood. System Design provides a clear architecture. Coding implements business logic. Testing identifies defects before release. Deployment delivers the product to users, while Maintenance keeps the software updated and secure. Following SDLC improves software quality, reduces failures, and increases customer trust.

Conclusion

The Software Development Life Cycle is essential for developing reliable, secure, and maintainable E-Commerce applications. By following every phase systematically, organizations can deliver high-quality software that satisfies customer requirements and supports future growth.

5. Selecting an Appropriate SDLC Model for Mobile App Development

Introduction

A Mobile Learning Application (Mobile Learning App) is designed to provide students with educational content, quizzes, video lectures, assignments, and progress tracking through smartphones or tablets. In many projects, customer requirements change frequently as new features are requested during development. Therefore, selecting the appropriate Software Development Life Cycle (SDLC) model is very important to ensure the project is completed successfully.

Agile Model

The Agile model is an iterative and incremental software development approach in which the software is developed in small cycles called sprints. After each sprint, a working version of the application is delivered, and customer feedback is collected. Based on the feedback, improvements and new features are added in the next iteration.

Advantages of Agile

- Easily accommodates changing requirements.
- Delivers working software quickly.
- Continuous customer involvement improves satisfaction.
- Frequent testing helps identify defects early.
- Supports teamwork and collaboration.
- Reduces development risks.

Limitations of Agile

- Requires experienced developers.
- Difficult to estimate the total project cost.
- Documentation may be less detailed.
- Frequent customer involvement is necessary.

Waterfall Model

The Waterfall model is a traditional sequential software development model. Each phase must be completed before moving to the next phase. Once a phase is completed, changes are difficult to implement.

Advantages of Waterfall

- Simple and easy to understand.
- Well-defined phases.
- Good documentation.
- Suitable for projects with stable requirements.

Limitations of Waterfall

- Difficult to accommodate requirement changes.

- Testing occurs after coding, making defect correction costly.
- Customer involvement is limited.
- Not suitable for dynamic projects.

Spiral Model

The Spiral model combines iterative development with risk analysis. Each cycle includes planning, risk assessment, development, and customer evaluation.

Advantages of Spiral

- Excellent risk management.
- Suitable for large and complex systems.
- Supports requirement changes.
- Continuous customer feedback.

Limitations of Spiral

- Expensive to implement.
- Requires skilled project managers.
- More time-consuming.
- Not suitable for small projects.



Comparison

Feature	Agile	Waterfall	Spiral
Development	Iterative	Sequential	Iterative
Requirement Changes	Easy	Difficult	Moderate

Feature	Agile	Waterfall	Spiral
Customer Feedback	Continuous	Limited	Continuous
Risk Handling	Moderate	Low	Excellent
Cost	Medium	Low	High

Recommendation

For a Mobile Learning Application where customer requirements change frequently, the **Agile model** is the most suitable SDLC model. Agile allows developers to respond quickly to changes, deliver software in small increments, and continuously improve the application based on user feedback.

Conclusion

Among Agile, Waterfall, and Spiral models, Agile provides the greatest flexibility, faster delivery, better customer involvement, and improved software quality. Therefore, Agile is the recommended SDLC model for mobile application development.

6. Modular Design in a Hospital Management System

Introduction

A Hospital Management System is a software application used to manage hospital operations such as patient registration, doctor scheduling, billing, pharmacy, laboratory, appointments, and medical records. If all these functions are tightly connected, the system becomes difficult to maintain and update. Object-oriented modular design solves these problems by dividing the system into independent modules.

Existing Problems

In the existing system, all functionalities are closely connected, creating several issues:

- Difficult to maintain.
- Changes in one module affect other modules.
- High code duplication.
- Difficult to test individual modules.
- Reduced scalability.
- Increased debugging effort.
- Longer development time.

Modular Design

The redesigned system divides the application into independent modules.

Patient Module

Handles patient registration, admission, discharge, and medical history.

Doctor Module

Manages doctor information, schedules, and consultation details.

Appointment Module

Schedules appointments between patients and doctors.

Laboratory Module

Stores laboratory reports and test results.

Pharmacy Module

Maintains medicine inventory and prescriptions.

Billing Module

Generates bills and manages payments.

Each module performs a specific task and communicates with other modules only when necessary.

Object-Oriented Principles Used

Encapsulation

Each module protects its internal data and exposes only required methods.

Inheritance

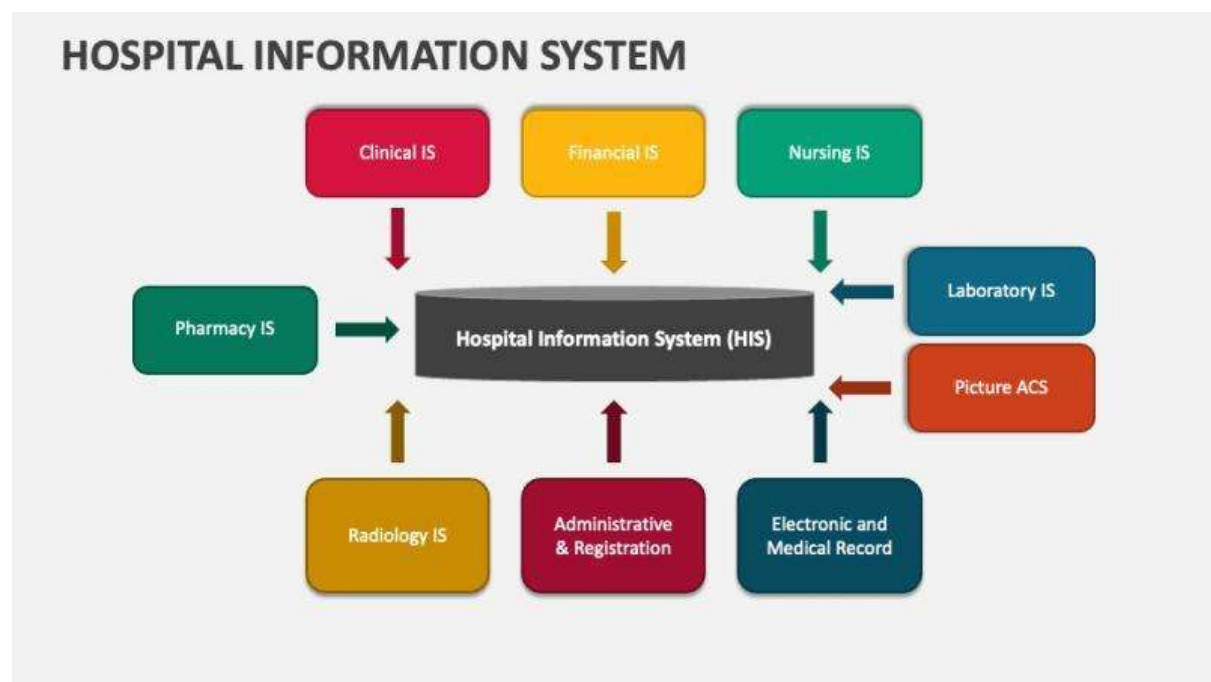
Common features are inherited from base classes, reducing duplicate code.

Polymorphism

Different billing or reporting methods can be implemented according to patient type.

Abstraction

Users interact only with necessary functions while internal processing remains hidden.



Advantages of Modular Design

- Easier maintenance.
- Independent testing.
- Better code reuse.
- Faster debugging.
- Improved scalability.
- Reduced development time.
- Better software quality.
- Simplified future enhancements.

Analysis

By separating the system into modules, developers can modify or upgrade one module without affecting others. This reduces maintenance cost and improves overall software reliability.

Conclusion

Modular design makes the Hospital Management System flexible, maintainable, scalable, and easier to manage. It supports future expansion and improves overall software performance.

7. Enhancing Software Quality Using Design Patterns

Introduction

A Hotel Reservation System manages room booking, customer registration, payment processing, room availability, and notifications. Poor software design often leads to duplicate code, inefficient object creation, and maintenance difficulties. Design Patterns provide reusable solutions to these common software design problems.

Existing Problems

- Duplicate code.
- Repeated object creation.
- Difficult maintenance.
- Poor flexibility.
- Increased development effort.

Factory Pattern

The Factory Pattern creates objects without exposing object creation logic to the client.

For example, when customers select different room types such as Standard Room, Deluxe Room, or Suite Room, the Factory Pattern creates the appropriate room object automatically.

Benefits

- Reduces code duplication.
- Simplifies object creation.

- Improves flexibility.
- Supports future room types.

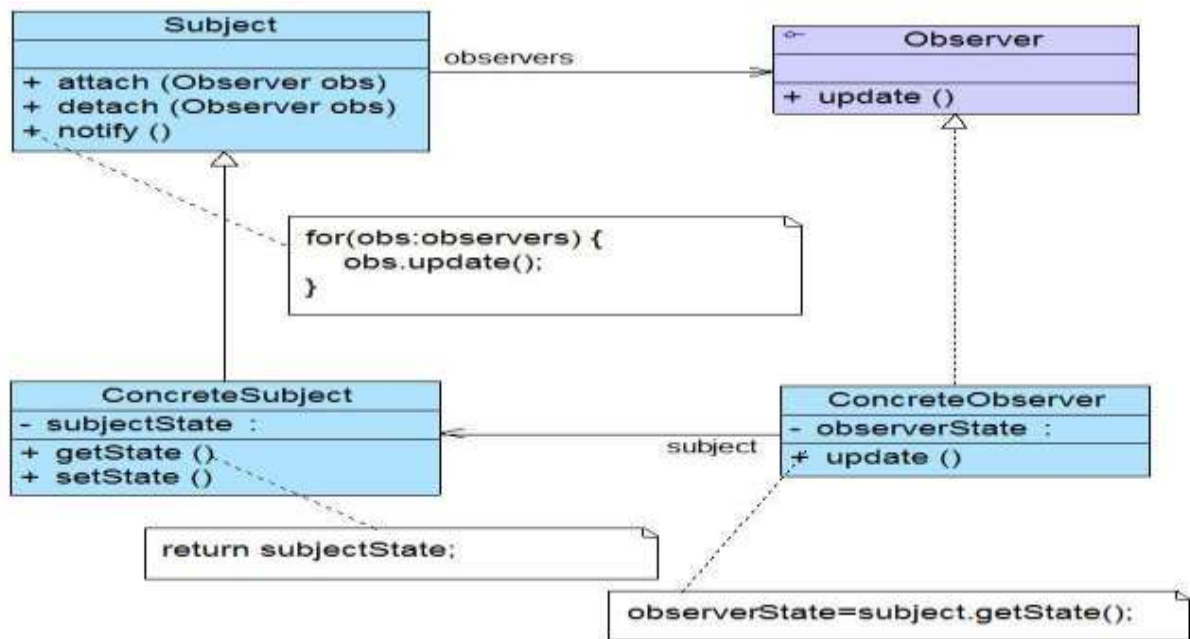
Singleton Pattern

The Singleton Pattern ensures that only one instance of a class exists throughout the application.

Example:

Database Connection Manager.

Instead of creating multiple database connections, the application uses a single shared connection.



Benefits

- Saves memory.
- Improves performance.
- Prevents inconsistent data.
- Simplifies resource management.

Observer Pattern

The Observer Pattern allows one object to notify multiple dependent objects whenever its state changes.

Example:

When a room booking is confirmed:

- Customer receives confirmation email.
- Hotel manager receives notification.
- Payment system updates transaction status.

Benefits

- Automatic notifications.
- Loose coupling.
- Easier maintenance.
- Improved communication among modules.

Overall Advantages

- Reusable code.
- Better flexibility.
- Easier maintenance.
- Reduced complexity.
- Improved scalability.
- Better software quality.

Analysis

Applying Factory, Singleton, and Observer patterns improves the software architecture by reducing redundancy, simplifying maintenance, and supporting future enhancements.

Conclusion

Design Patterns provide standardized solutions to common software design problems. They increase software quality, improve maintainability, and make Hotel Reservation Systems more efficient and scalable.

8. UML-Based Modeling for an Online Examination System

Introduction

An Online Examination System is a software application used to conduct examinations through the internet. It manages students, instructors, examinations, question papers, result processing, and report generation. Unified Modeling Language (UML) helps visualize and design the entire system before development begins.

Use Case Diagram

The Use Case Diagram represents the interaction between users and the system.

Actors

- Student
- Instructor
- Administrator

Main Functions

- Login
- Register

- Create Examination
- Upload Questions
- Attend Examination
- Submit Answers
- Evaluate Answers
- View Results
- Generate Reports

The Use Case Diagram helps developers understand the system's functional requirements.

Class Diagram

The Class Diagram represents the static structure of the system.

Main Classes

- Student
- Instructor
- Examination
- Question
- Result
- Administrator

Each class contains attributes and methods that define its properties and behaviors.

The Class Diagram helps developers understand object relationships and software architecture.

Activity Diagram

The Activity Diagram represents the workflow of the examination process.

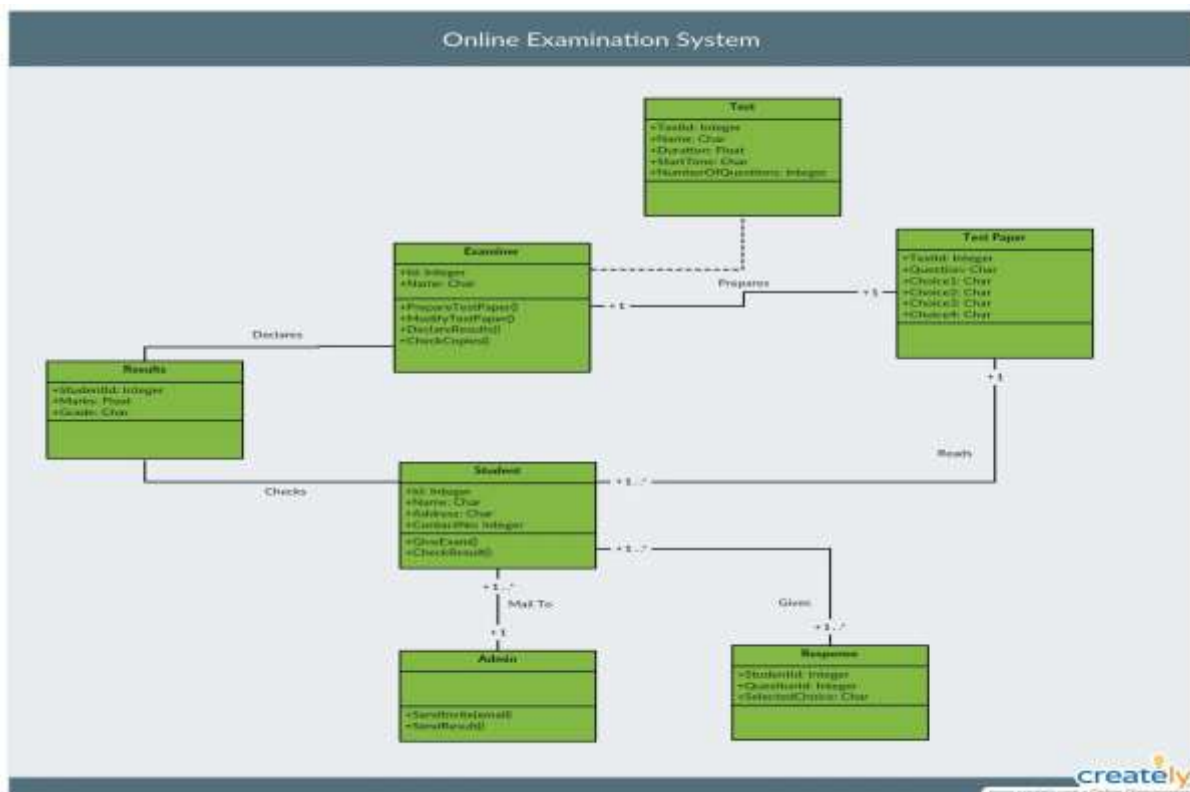
Typical flow:

1. Student Login
2. Select Examination
3. Read Instructions
4. Start Examination
5. Answer Questions
6. Submit Examination
7. Automatic Evaluation
8. Display Results

The Activity Diagram clearly represents system behavior and process flow.

Importance of UML Modeling

- Improves system visualization.
- Simplifies requirement analysis.
- Enhances communication among developers and stakeholders.
- Detects design issues before coding.
- Improves software documentation.
- Supports object-oriented development.
- Reduces development cost.
- Simplifies testing and maintenance.



Analysis

UML diagrams provide a clear representation of system structure, behavior, and interactions. They help developers understand project requirements, reduce design errors, and improve communication throughout the software development lifecycle.

Conclusion

UML modeling is an essential part of software engineering. It helps developers build reliable, maintainable, and scalable Online Examination Systems by providing a clear blueprint before implementation.